Product Inventory Management System

Team Information

Team ID: SWTID-2026-2416

Team Size: 4

Team Leader: PAVITHRA G

Team Member: THAMARAISELVI N

Team Member: SWARANALEKHA S

Team Member: REVATHI

1. Introduction

This project is a backend-based web application developed using Node.js, Express.js, and MongoDB.

It allows users to manage product inventory securely using REST APIs. Users can register, log in,

and perform operations like adding, updating, viewing, and deleting products.

2. Project Overview

The Product Inventory Management System helps users manage product data in an organized way.

Each user can maintain their own inventory and access only their products.

Main Features:

• User registration and login

• Secure JWT authentication

• Add products

• View products

• Update product details

• Delete products

• Search products

3. Architecture

Backend: Node.js + Express.js

Database: MongoDB

Authentication: JWT (JSON Web Token)

Architecture Pattern: MVC (Model View Controller)

Components:

• Controllers – Handle business logic

• Models – Define database schema

• Routes – API endpoints

• Middleware – Authentication verification

4. Setup Instructions

Prerequisites:

• Node.js

• MongoDB

• Visual Studio Code

Installation Steps:

1. Create a project folder

2. Open the folder in VS Code

3. Open terminal and run:

```
npm init -y
```

4. Install dependencies:

```
npm install express mongoose cors dotenv jsonwebtoken bcryptjs
```

5. Create .env file:

```
PORT=5000
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_secret_key
```

6. Start the server:

```
npm start
```

5. Folder Structure

```
server/
│
├── node_modules/
├── src/
│   ├── controllers/
│   ├── db/
│   ├── middlewares/
│   ├── models/
│   ├── routes/
│   └── server.js
```

```
│
├── .env
├── package.json
└── package-lock.json
```

6. Running the Application

Start the server using:

```
npm start
```

Server will run on:

http://localhost:5000

7. API Documentation

Authentication APIs

POST /api/v1/auth/register

POST /api/v1/auth/login

Product APIs

POST /api/v1/products

GET /api/v1/products

PUT /api/v1/products/:id

DELETE /api/v1/products/:id

GET /api/v1/products/search

8. Authentication

The system uses JWT authentication.

Process:

1. User registers or logs in

2. Server generates JWT token

3. Token is sent in Authorization header

4. Middleware verifies token

5. Protected APIs allow access

9. Testing

APIs can be tested using:

• Thunder Client (VS Code)

• Postman

Tested operations:

• User registration

• User login

• Create product

• Get products

• Update product

• Delete product

10. Future Enhancements

• Add frontend using React.js

• Implement product categories

• Add low stock alerts

• Create admin dashboard

- Deploy project to cloud

API Testing Screenshots

Figure 1: User Registration API

```
export const registerUser = async (req, res) => {
  try {…
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};


export const loginUser = async (req, res) => {
  try {…
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

Figure 2: User Login API

```
const authRoutes = express.Router();


authRoutes.post("/register", registerUser);

authRoutes.post("/login", loginUser);


export default authRoutes;
```

Figure 3: Create Product API

```
export const createProduct = async (req, res) => {
  try {…
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

export const getProducts = async (req, res) => {
  try {…
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

export const updateProduct = async (req, res) => {
  try {…
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

export const deleteProduct = async (req, res) => {
  try {…
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

Figure 4: Get Products API

```
27
28   app.use("/api/v1/auth", authRoutes);
29   app.use("/api/v1/products", productRoutes);
30
```

Figure 5: Update/Delete Product API

GET http://localhost:5000/api/v1/products

Docs Params Authorization Headers (7) Body Scripts Settings

Body Cookies Headers (10) Test Results 200 OK

JSON Preview Visualize


```
[
    {
        "_id": "6995954ef9677779f15b95a4",
        "name": "iPhone 15",
        "price": 90000,
        "quantity": 5,
        "description": "Apple mobile phone",
        "userId": "6992994965747f02ccabc5a7",
        "createdAt": "2026-02-18T10:32:46.935Z",
        "updatedAt": "2026-02-18T10:32:46.935Z",
        "__v": 0
    }
]
```